

DATA FORT

Personal Digital Vault

FULL TECHNICAL STACK DOCUMENT

Final submission-ready technology and architecture reference

This document records the actual technologies, versions, architecture, modules, security mechanisms, database design, API surface, and supporting tools present in the final stable Data Fort implementation.

Edition	Final stable version documentation
Review date	14 September 2026
Backend target	.NET 10.0 / ASP.NET Core Web API
Frontend	Angular 21.2.x / TypeScript 5.9.x
Database	Microsoft SQL Server / EF Core 10.0.11
Local API	https://localhost:7166/api
Local UI	http://localhost:4200

Prepared from the supplied final stable ZIP and the previous Data Fort run guide. Version-sensitive details were reconciled against the actual project files and package-lock.json; no unimplemented BRD feature has been added.

1. Document Scope and Source-of-Truth

This is the final technical stack reference for the stable Data Fort repository. The implementation was reviewed from the supplied ZIP, including the solution, C# project file, Angular package.json/package-lock.json, application configuration, EF Core migrations, controllers, services, models, routing, and storage folders.

- Backend target framework: net10.0 (ASP.NET Core Web API).
- Backend direct package versions are taken from PersonalDigitalVault.API.csproj.
- Frontend resolved package versions are taken from package-lock.json; declared ranges remain shown where useful.
- The final database model contains 9 application tables.
- The final source uses SMTP email delivery; no OpenAI, Azure, Google Maps, cloud storage SDK, or other external API integration was found in the stable source.

Submission requirement mapping

Required item	Final Data Fort value
.NET Version	.NET 10.0 / target framework net10.0; ASP.NET Core Web API
Frontend Technology / Framework	Angular 21.2.x with standalone bootstrap and Angular Router
Backend Technology / Framework	ASP.NET Core Web API on .NET 10.0
Database	Microsoft SQL Server - database: PersonalDigitalVaultDB
Programming Languages	C#, TypeScript, HTML, CSS
APIs / Services used	REST API, JWT Bearer auth, SMTP email, Swagger/OpenAPI
Other Tools / Libraries	EF Core, SQL Server provider, Swashbuckle, Tailwind CSS, RxJS, Vitest, Prettier, PostCSS, Visual Studio, SSMS, Git/GitHub

2. System Architecture

Data Fort follows a layered full-stack architecture. The Angular SPA is the presentation layer. It calls ASP.NET Core Web API controllers through Angular HttpClient. Controllers validate and route requests to module services. Repository classes use ApplicationDbContext to persist domain data in SQL Server. File storage is local to the API process under the Storage/ProtectedFiles/Documents folder.

ANGULAR 21 SPA	ASP.NET CORE WEB API	SQL SERVER
Pages + Components Services + Models Router + Guards HTTP interceptor	Controllers DTOs + Validators Application Services Repositories Security + File Services	Roles Users Vault data Sharing Audit logs EF Core migrations

Request flow

Browser

- > Angular component
- > Angular feature service (HttpClient)
- > authInterceptor adds Bearer token when available
- > HTTPS REST endpoint under /api
- > Controller
- > DTO / validation
- > Service layer
- > Repository / security / storage service
- > Entity Framework Core
- > SQL Server / local protected file storage
- > response back to Angular

Dependency injection

Program.cs registers repositories, application services, validators, JWT helpers, email service, encryption service, hashing service, file storage service, and admin/public-sharing services with ASP.NET Core dependency injection.

3. Backend Technology Stack

Technology	Version / implementation	Role in Data Fort
.NET / ASP.NET Core	Target framework net10.0	Web API runtime, dependency injection, configuration, HTTP pipeline, authorization.
Entity Framework Core	10.0.11	ORM, entity mapping, change tracking, migrations, database access.
SQL Server provider	10.0.11	EF Core provider for SQL Server.
JWT Bearer	10.0.11	Access-token validation and secured API endpoints.
ASP.NET Core OpenAPI	10.0.11	OpenAPI document generation support.
Swashbuckle.AspNetCore	10.2.3	Swagger/OpenAPI generation and interactive Swagger UI.
PasswordHasher	ASP.NET Core Identity API	Password hashing and password verification.
System.Security.Cryptography	.NET platform APIs	AES encryption, SHA-256 hashing, cryptographically secure token generation.
System.Net.Mail	.NET platform API	SMTP email verification and password-reset messages.

Backend module layout

Module	Main responsibilities
Authentication	Register, email verification, login, forgot/reset password, change password, profile view/update.
Secure Vault	Folders, nested folders, documents, encrypted file handling, credentials, search, integrity verification.
Public Sharing	Create/manage share links, expiry, revocation, public metadata view, public download.
Administration	Dashboard statistics, user listing, user activation/deactivation, audit logs.
Data / Infrastructure	ApplicationDbContext, DbInitializer, EF Core migrations, repositories, local protected-file storage.

Repository pattern

The backend separates controller, service, and repository responsibilities. Controllers handle HTTP-level concerns, services contain application/business logic, and repositories query or update EF Core entities through ApplicationDbContext.

4. Backend Direct Packages

Package	Version	Type	Purpose
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.11	Runtime	JWT Bearer authentication and token validation.
Microsoft.AspNetCore.OpenApi	10.0.11	Runtime	OpenAPI support and endpoint metadata.
Microsoft.EntityFrameworkCore.Design	10.0.11	Design-time	EF Core design-time functionality for migrations and schema development.
Microsoft.EntityFrameworkCore.SqlServer	10.0.11	Runtime	SQL Server database provider for EF Core.
Microsoft.EntityFrameworkCore.Tools	10.0.11	Design-time	EF Core tooling for migration/database operations.
Swashbuckle.AspNetCore	10.2.3	Runtime / Docs	Swagger generator and interactive Swagger UI.

Configuration model

Configuration key	Purpose	Source / handling
ConnectionStrings:DefaultConnection	SQL Server connection to PersonalDigitalVaultDB	appsettings.json
Jwt:Issuer	JWT issuer validation	appsettings.json
Jwt:Audience	JWT audience validation	appsettings.json
Jwt:ExpiryMinutes	Configured token lifetime value	appsettings.json
Jwt:Key	JWT signing secret	User Secrets / secure configuration
DocumentEncryption:Key	AES document/credential encryption key	User Secrets / secure configuration
DocumentEncryption:KeyId	Encryption key identifier persisted with documents	User Secrets / secure configuration
Smtp:Host / Port / User / Password	Email verification and password-reset delivery	User Secrets / secure configuration

Security note: actual secret values are intentionally not reproduced in this public-facing PDF. The supplied project package contains USER SECRETS.docx; that file should be treated as sensitive and must not be committed to a public GitHub repository.

5. Frontend Technology Stack

Technology	Resolved version	Role
@angular/core / common / compiler / forms / platform-browser / router	21.2.22	Angular framework, browser integration, forms, and SPA navigation.
@angular/build	21.2.23	Application build tooling.
@angular/cli	21.2.23	Angular CLI commands such as ng serve and ng build.
@angular/compiler-cli	21.2.22	Angular compiler tooling.
TypeScript	5.9.3	Frontend language, type checking, compilation.
RxJS	7.8.2	Observable-based asynchronous data flow.
Tailwind CSS	4.3.3	Utility-first UI styling.
@tailwindcss/postcss	4.3.3	Tailwind CSS integration with PostCSS.
PostCSS	8.5.28	CSS processing in the frontend build pipeline.
zone.js	0.16.3	Angular async/change-detection integration.
tslib	2.8.1	TypeScript runtime helper library.
jsdom	28.1.0	Browser-like DOM environment for tests.
Vitest	4.1.11	Frontend unit-test runner.
Prettier	3.9.6	Source formatting.

Angular application style

- Standalone bootstrap via bootstrapApplication(App, appConfig).
- provideRouter() supplies client-side SPA navigation.
- provideHttpClient(withInterceptors([authInterceptor])) configures HTTP and the JWT interceptor.
- Lazy-loaded feature routes divide Authentication, Secure Vault, Public Sharing, and Administration.
- Reactive Forms are used in authentication and vault entry forms.
- Core guards protect authenticated and administrator routes.

Effective API configuration

```
src/environments/environment.ts
apiUrl: 'https://localhost:7166/api'
```

```
Angular dev server
http://localhost:4200
```

The repository also contains environment.development.ts with an older localhost:5000 API value, but the current angular.json does not configure a file replacement for it. The effective runtime value used by the current app is therefore environment.ts.

6. Frontend Feature Architecture and Routes

Feature	Frontend routes / pages	Protection
Authentication	/auth/login, /auth/register, /auth/forgot-password, /auth/reset-password, /auth/verify-email, /auth/change-password, /auth/profile	Change password/profile use authGuard.
Secure Vault	/vault, /vault/folders, /vault/folders/new, /vault/folders/edit/:id, /vault/documents, /vault/documents/upload, /vault/documents/:id, /vault/credentials, /vault/credentials/new, /vault/credentials/edit/:id, /vault/search	Parent user layout uses authGuard.
Public Sharing	/sharing/manage; public file view at /public/file/:token	Management is under authenticated user layout; public file route is unauthenticated.
Administration	/admin/dashboard, /admin/users, /admin/audit-logs	authGuard + adminGuard.
Error pages	/forbidden, wildcard -> not-found	Public.

Core frontend services

Layer	Components present
Core	auth.guard.ts, admin.guard.ts, auth.interceptor.ts, token.service.ts, navigation.service.ts
Authentication feature	auth.service.ts plus login/register/verify/reset/change-password/profile pages
Secure Vault feature	folder, document, credential, and search services with their pages/components
Public Sharing feature	public-sharing.service.ts, share-link management, public-file page
Administration feature	admin-dashboard.service.ts, admin-user.service.ts, admin-audit-log.service.ts
Shared UI	navbar, sidebar, loading, empty-state, confirm-dialog, session-expired, auth-scene, file-size pipe

7. Database and Entity Design

Database name: **PersonalDigitalVaultDB**. SQL Server is accessed through EF Core 10.0.11. The current model has 9 application tables and three migrations.

Table	Primary key	Important relationships / purpose
Roles	RoleId	One role to many users; unique RoleName.
Users	UserId	Belongs to Role; parent for folders, documents, credentials, share links, audit logs, token tables; unique Email and UserName.
PasswordResetTokens	TokenId	Many tokens per user; TokenHash stored instead of raw reset token.
EmailVerificationTokens	TokenId	Many tokens per user; unique TokenHash; used/expiry tracking.
Folders	FolderId	Belongs to User; optional self-referencing ParentFolder; contains Documents and Credentials.
Documents	DocumentId	Belongs to User and optional Folder; links to ShareLinks; stores encryption metadata and SHA-256 hash.
Credentials	CredentialId	Belongs to User and optional Folder; encrypted password field.
ShareLinks	ShareLinkId	Belongs to User and Document; unique ShareToken; expiry and revocation fields.
AuditLogs	AuditLogId	Optional User reference; records action/entity/IP/time details.

Database constraints and integrity rules

- Roles.RoleName, Users.Email, and Users.UserName are unique.
- EmailVerificationTokens.TokenHash and ShareLinks.ShareToken are unique.
- Root folders and root documents use filtered unique indexes to prevent active duplicates by normalized name.
- Child folders and child documents use parent-aware filtered unique indexes.
- Folder self-reference supports nested folder structures.
- Foreign-key delete behaviors are deliberately restrictive/no-action to avoid unintended cascading deletes.

EF Core migrations in the repository

Migration	Purpose
InitialCreate	Initial application database schema.
AddEmailVerificationFields	Adds email verification state fields to users.
AddEmailVerificationTokenTable	Adds the EmailVerificationTokens table and relationships.

8. Security Implementation

Security control	Implementation in the final source
Authentication	JWT Bearer access token generated after successful login; secured APIs require [Authorize].
Authorization	Administrator APIs require [Authorize(Roles = "Administrator")]; Angular adminGuard protects admin UI routes.
Password storage	ASP.NET Core PasswordHasher stores password hashes; raw passwords are not persisted.
Credential password protection	Credential passwords are encrypted by EncryptionService before database storage and decrypted when needed for an authenticated user.
Document encryption	Uploaded files are encrypted on local disk using the configured AES key and a fresh 16-byte IV; IV and key identifier are stored as document metadata.
Document integrity	SHA-256 hash is computed from document content and stored; verify-integrity recomputes/compares the hash.
Email verification	A random token is generated, SHA-256 hashed for storage, expires after 15 minutes, and the raw token is sent only in the email link.
Password reset	A random token is generated, SHA-256 hashed for storage, expires after 15 minutes, and becomes used after successful reset.
Public sharing	Share tokens are unique; links support expiration and revocation. Public access is token-based and separate from authenticated vault APIs.
CORS	Frontend origin http://localhost:4200 is explicitly allowed in the API CORS policy.
HTTPS	The configured https launch profile serves https://localhost:7166; UseHttpsRedirection() is enabled.

JWT claims

The project documentation/source indicates tokens carry user identity, email, username, and role claims. API authorization reads the identity/role claims to enforce user ownership and administrator access.

9. REST API Surface

Area	Method	Endpoint	Access
Authentication	POST	/api/auth/register	Public
Authentication	POST	/api/auth/login	Public
Authentication	POST	/api/auth/forgot-password	Public
Authentication	POST	/api/auth/reset-password	Public
Authentication	POST	/api/auth/change-password	Authenticated
Authentication	GET	/api/auth/profile	Authenticated
Authentication	PUT	/api/auth/profile	Authenticated
Authentication	GET	/api/auth/verify-email	Public
Folders	GET	/api/folders	Authenticated
Folders	POST	/api/folders	Authenticated
Folders	PUT	/api/folders/{id}	Authenticated
Folders	DELETE	/api/folders/{id}	Authenticated
Documents	GET	/api/documents	Authenticated
Documents	POST	/api/documents	Authenticated
Documents	PUT	/api/documents/{id}	Authenticated
Documents	DELETE	/api/documents/{id}	Authenticated
Documents	POST	/api/documents/upload	Authenticated
Documents	GET	/api/documents/{id}/download	Authenticated
Documents	POST	/api/documents/{id}/verify-integrity	Authenticated
Credentials	GET	/api/credentials	Authenticated
Credentials	POST	/api/credentials	Authenticated
Credentials	PUT	/api/credentials/{id}	Authenticated
Credentials	DELETE	/api/credentials/{id}	Authenticated
Search	GET	/api/search?searchTerm=...	Authenticated
Sharing	POST	/api/share-links	Authenticated
Sharing	GET	/api/share-links	Authenticated
Sharing	POST	/api/share-links/{id}/revoke	Authenticated
Sharing	PUT	/api/share-links/{id}	Authenticated
Sharing	DELETE	/api/share-links/{id}	Authenticated
Public sharing	GET	/api/public/share/{token}	Public
Public sharing	GET	/api/public/share/{token}/download	Public
Administration	GET	/api/admin/dashboard	Administrator
Administration	GET	/api/admin/users	Administrator
Administration	PUT	/api/admin/users/{id}/status	Administrator
Administration	GET	/api/admin/audit-logs	Administrator

10. Tools, Local Development, and Deployment Scope

Tool / technology	Use in the project
Visual Studio	Primary .NET development environment referenced by the project workflow.
SQL Server / SQL Server Express / LocalDB	Local SQL Server engine options for PersonalDigitalVaultDB.
SQL Server Management Studio (SSMS)	Database inspection, restore, and administration.
Node.js + npm	Angular dependency installation and frontend development server.
Git + GitHub	Source control and repository hosting.
Swagger UI	Interactive REST endpoint testing during Development.
EF Core migrations	Schema creation and update workflow.
Local file system	Encrypted document storage under API Storage/ProtectedFiles/Documents.
SMTP	Email verification and password-reset email delivery.

Repository structure

```

DATA FORT FINAL STABLE VERSION/
|-- PersonalDigitalVault.API/
|   |-- Administration/
|   |-- Authentication/
|   |-- Data/
|   |-- Migrations/
|   |-- Models/
|   |-- PublicSharing/
|   |-- Repositories/
|   |-- SecureVault/
|   |-- Storage/ProtectedFiles/Documents/
|   |-- Program.cs
|   |-- appsettings.json
|   |-- Properties/launchSettings.json
|-- PersonalDigitalVault.Frontend/
|   |-- src/app/core/
|   |-- src/app/features/
|   |-- src/app/layouts/
|   |-- src/app/pages/
|   |-- src/app/shared/
|   |-- package.json
|   |-- package-lock.json
|   |-- angular.json
|-- PersonalDigitalVault.sln
|-- DIGITAL VAULT.bak
|-- Data_Fort_ER_Diagram.pdf

```

Deployment scope note

The final stable implementation is configured and documented for local development/testing. The Angular frontend points to the local HTTPS API and the backend stores encrypted documents on the local file system. No cloud deployment target or cloud storage SDK is wired into this final source.

Final technical stack statement

Data Fort is a .NET 10 / ASP.NET Core Web API + Angular 21 single-page application using EF Core 10.0.11 and SQL Server, with JWT authentication, role-based authorization, SMTP email flows, AES-protected vault data, SHA-256 document integrity verification, local encrypted file storage, public tokenized sharing, Swagger/OpenAPI, Tailwind CSS, RxJS, Vitest, and a repository/service layered architecture.